

TechNova Solutions Pvt. Ltd.

IT & Software Engineering Handbook — 2025–26

This is a fictional company handbook created for Enterprise RAG testing. All company names, products, teams, technologies, processes, and examples are synthetic.

1. Company Technology Overview

TechNova Solutions Pvt. Ltd. is a fictional enterprise software company that develops cloud-based business products, artificial intelligence platforms, data analytics systems, workflow automation tools, cybersecurity solutions, and custom enterprise applications.

The technology organization is responsible for product engineering, infrastructure, quality assurance, cybersecurity, data engineering, artificial intelligence, and technical support.

2. Types of Products Developed

Product Category	Description
SaaS Platforms	Cloud applications used by businesses for operations, collaboration, and analytics.
AI Products	AI assistants, document analysis, enterprise search, and predictive analytics systems.
Data Platforms	Data pipelines, dashboards, reporting systems, and business intelligence tools.
Workflow Automation	Software that automates approvals, notifications, repetitive business processes, and internal workflows.
Cybersecurity Products	Security monitoring, identity management, audit, and threat detection solutions.
Mobile Applications	Android and iOS applications supporting customers and internal operations.
Enterprise APIs	REST and event-driven APIs used to integrate company products with external systems.
Cloud Infrastructure	Internal platforms and services used to deploy and operate software products.

3. Software Engineering Organization

Team	Primary Responsibilities
Product Management	Product strategy, requirements, roadmap, customer needs, and prioritization.
Backend Engineering	APIs, business logic, databases, integrations, and backend services.
Frontend Engineering	Web interfaces, dashboards, user experience, and frontend applications.
Mobile Engineering	Android and iOS applications and mobile-specific services.
AI/ML Engineering	Machine learning models, LLM applications, RAG systems, and AI services.
Data Engineering	Data pipelines, warehouses, ETL/ELT processes, and analytics infrastructure.
QA Engineering	Functional, integration, regression, performance, and automated testing.
DevOps / Platform	CI/CD, containers, Kubernetes, cloud infrastructure, and deployment automation.
Security Engineering	Application security, identity, vulnerability management, and security monitoring.
SRE	Reliability, availability, observability, incident response, and production operations.
Technical Support	Customer technical issues, troubleshooting, escalation, and incident communication.

4. Common Software Engineering Roles

- Software Engineer: Designs, implements, tests, and maintains software features.
- Senior Software Engineer: Leads complex technical work and mentors engineers.
- Tech Lead: Coordinates technical decisions and architecture within a team.
- Engineering Manager: Manages people, delivery, team priorities, and engineering processes.
- Product Manager: Defines product requirements, priorities, and roadmap.
- QA Engineer: Designs and executes tests and helps prevent defects.
- DevOps Engineer: Automates build, deployment, infrastructure, and operational workflows.
- Cloud Engineer: Designs and manages cloud resources and cloud architecture.
- SRE: Improves system reliability, observability, scalability, and incident response.
- Data Engineer: Builds data pipelines and data processing systems.
- ML/AI Engineer: Develops and integrates machine learning and AI systems.
- Security Engineer: Identifies and mitigates application and infrastructure security risks.

5. Software Development Lifecycle

- Requirement Analysis: Product managers and engineers clarify the business problem and acceptance criteria.
- Design: Engineers create technical designs, API contracts, database models, and architecture plans.
- Implementation: Developers create the feature in a dedicated Git branch.
- Code Review: At least one qualified engineer reviews the change before it is merged.
- Automated Testing: Unit, integration, and other applicable automated tests run through CI.
- QA Validation: QA verifies functionality against requirements and regression risks.
- Staging Deployment: Approved changes are deployed to a staging environment for final validation.
- Production Deployment: Changes are released using the approved deployment process.
- Monitoring: Logs, metrics, traces, alerts, and user feedback are monitored after release.

6. Git and Code Management

- The main branch contains production-ready code.
- Developers create feature or bug-fix branches for changes.
- Commit messages should clearly describe the change.
- Pull requests must contain a useful description and relevant testing information.
- Code review is required before production code is merged.
- Secrets, passwords, API keys, and private credentials must never be committed to source control.

7. Testing and Quality Assurance

- Unit tests validate individual functions and components.
- Integration tests validate interactions between services, databases, and external systems.
- End-to-end tests validate complete user workflows.

- Regression testing is performed when changes could affect existing functionality.
- Performance testing is required for systems with significant scalability requirements.
- Security testing is performed for applications handling sensitive or critical information.

8. DevOps and CI/CD

- Continuous Integration automatically builds and tests software after approved code changes.
- Continuous Delivery or Deployment moves validated software toward production using automated pipelines.
- Docker containers are used for consistent application packaging where appropriate.
- Infrastructure should be managed through repeatable configuration or infrastructure-as-code practices.
- Production deployments should be traceable to a specific source-code version.
- Rollback procedures must exist for critical production services.

9. Cloud and Infrastructure

- Production workloads are deployed using approved cloud infrastructure and internal platform services.
- Infrastructure resources should follow least-privilege and network segmentation principles.
- Production and development environments should be logically separated.
- Backups must be configured for critical databases and services.
- Infrastructure capacity should be monitored and scaled according to demand.

10. Application Security

- Authentication and authorization must be implemented for protected resources.
- Applications must validate and sanitize untrusted input.
- Sensitive credentials must be stored using approved secret-management mechanisms.
- Dependencies should be regularly checked for known vulnerabilities.
- Security incidents must be reported immediately to the Security Engineering team.
- Access to production systems must be limited to authorized personnel.

11. AI and Machine Learning Systems

- AI systems must be evaluated for accuracy, reliability, security, and appropriate use.
- RAG systems should apply document access controls before retrieving enterprise content.
- Training or evaluation data must be handled according to applicable company policies.
- LLM outputs should not be treated as authoritative for high-impact decisions without appropriate human review.
- AI services should be monitored for latency, failures, quality, and unexpected behavior.

12. Incident Management

- Production incidents are classified according to business impact and service availability.
- The on-call engineer or SRE begins initial investigation for production incidents.
- Critical incidents require timely communication to relevant stakeholders.
- Engineers should document the timeline, impact, root cause, and remediation actions.
- A post-incident review should identify improvements that reduce the chance of recurrence.

13. Monitoring and Observability

- Production services should provide appropriate logs, metrics, and health checks.
- Critical services should have alerts for availability, latency, errors, and resource exhaustion.
- Distributed systems should use tracing where required to diagnose cross-service failures.
- Monitoring dashboards should provide enough information for engineers to investigate common incidents.

14. Documentation Standards

- Every production service should have basic setup and deployment documentation.
- Public or internal APIs should document endpoints, authentication, request formats, and response formats.
- Architecture decisions should be recorded when they have significant long-term impact.
- Runbooks should describe common operational procedures and incident recovery steps.

15. Agile and Team Collaboration

- Engineering teams normally work in two-week development iterations.
- Sprint planning defines work based on product priorities and engineering capacity.
- Daily stand-ups are used to coordinate progress and identify blockers.
- Sprint reviews demonstrate completed work to stakeholders.
- Retrospectives identify process improvements for future iterations.
- Cross-functional collaboration between product, engineering, design, QA, security, and operations is encouraged.

16. Example Technology Stack

Area	Example Technologies
Frontend	React, TypeScript, Vite
Backend	Python, FastAPI, Node.js
Databases	PostgreSQL, Redis
AI / RAG	LLMs, embeddings, vector databases, retrieval pipelines
Containers	Docker
Orchestration	Kubernetes
CI/CD	GitHub Actions or GitLab CI
Cloud	AWS, Azure, or Google Cloud
Observability	Prometheus, Grafana, OpenTelemetry
Version Control	Git and GitHub/GitLab

17. Suggested Enterprise RAG Questions

- What types of products does TechNova develop?
- What responsibilities does the backend engineering team have?
- What is the role of an SRE?
- Who is responsible for product requirements?

- What happens during the code review stage?
- What is the difference between unit and integration testing?
- What tools are used for containerization?
- How does TechNova handle production deployments?
- What information should be documented for an API?
- Who handles production incidents?
- What security practices are required for application development?
- What technologies are used for the RAG platform?
- How often do engineering teams work in development iterations?
- What is the purpose of a sprint retrospective?
- What monitoring information should production services provide?
- How should an AI/RAG system enforce document access controls?

Document classification: Synthetic / Dummy Data. Created specifically for Enterprise RAG testing.